ASI Aventure
partie 5

Remarque

— pour chaque classe référez vous à la javadoc
— limitez vous à ce que vous avez vu en cours
— méthode toString() à faire systématiquement

1 Passage aux collections

L'objectif de cette section est de continuer à passer vos tests unitaires tout en modifiant la structure interne de vos classes.

> — vous ne devez pas modifier les tests unitaires
> — vous ne devez pas modifier les signatures du jeu d'aventure

Remplacez votre gestion de tableaux par des classes liées aux collections. Choisissez les classes en fonction des besoins de manipulation des données. Restez le plus générique possible sur vos déclarations de types en utilisant des types abstraits ou des interfaces.

Pour chaque classe modifiée, vous devez :

compiler et lancer tous les tests
Tant que `AllTests` ne passe pas, on ne passe pas à la suite !

Modifiez vos classes dans cet ordre :

1. `Monde`
2. `Vivant`
3. `Piece`

2 Changer les retours de tableaux

Certaines méthodes des classes modifiées retournent des tableaux, vous allez modifier les signatures. Pour chaque classe et toujours dans l'ordre ci-dessus, effectuez les étapes suivantes :

1. commentez la méthode
2. réécrire la méthode avec une `Collection` paramétrée en retour
3. modifiez les tests unitaires nécessaires
4. **tant que `AllTests` ne passe pas, on ne passe pas à la suite !**
5. à la lecture de la documentation liées à `values()`, que devez vous modifier

3 La classe `Porte`

Créez la classe `Porte` et ses méthodes mais en excluant tout ce qui est gestion des serrures. Pour cela, regardez le diagramme des classes et la javadoc.

Par défaut, la porte :

— est fermée ;
— n'est pas activable avec un objet ;
— lance l'exception `ActivationImpossibleAvecObjetException` si elle est activée avec un objet

Testez la classe `Porte`

4 Gestion des portes dans les pièces

Modifiez le code de `Piece` afin de prendre en compte la gestion de `Porte`.
Testez la gestion des `Porte` dans les pièces.

5 Gestion des portes dans les vivants

Modifiez le code de `Vivant` afin de prendre en compte la gestion de `Porte`.
Prenez en compte la gestion des exceptions liées aux portes : `PorteFermeException` et `PorteInexistanteDansLaPieceException`

Testez le franchissement des portes.